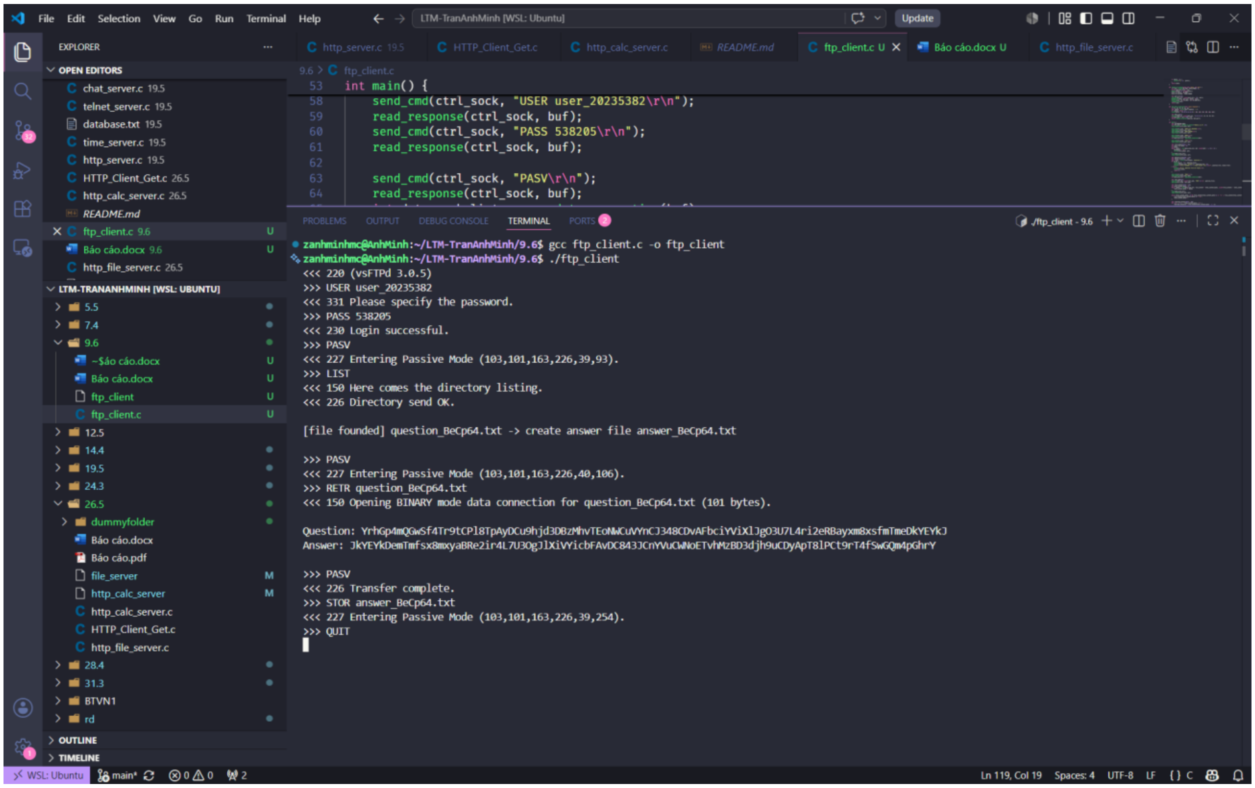
ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

BÁO CÁO BÀI TẬP
LẬP TRÌNH MẠNG

Trần Anh Minh - 20235382

HÀ NỘI, 06/2026

Ảnh khi chạy chương trình



Source code

```c
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <arpa/inet.h>
#include <unistd.h>
#include <netdb.h>

#define BUFFER_SIZE 4096

void send_cmd(int sock, const char *cmd) {
    send(sock, cmd, strlen(cmd), 0);
    printf(">>> %s", cmd);
```

```c
}

int read_response(int sock, char *buffer) {
    memset(buffer, 0, BUFFER_SIZE);
    int bytes = recv(sock, buffer, BUFFER_SIZE - 1, 0);
    if (bytes > 0) {
        printf("<<< %s", buffer);
    }
    return bytes;
}

int connect_to_server(const char *host, int port) {
    int sock = socket(AF_INET, SOCK_STREAM, 0);
    struct addrinfo hints, *res;
    memset(&hints, 0, sizeof(hints));
    hints.ai_family = AF_INET;
    hints.ai_socktype = SOCK_STREAM;

    char port_str[10];
    snprintf(port_str, sizeof(port_str), "%d", port);
    getaddrinfo(host, port_str, &hints, &res);
    connect(sock, res->ai_addr, res->ai_addrlen);
    freeaddrinfo(res);
    return sock;
}

int open_data_connection(char *pasv_response) {
    int h1, h2, h3, h4, p1, p2;
    char *paren = strchr(pasv_response, '(');
    if (!paren) return -1;
    sscanf(paren, "(%d,%d,%d,%d,%d,%d)", &h1, &h2, &h3, &h4, &p1, &p2);
```

```c
    char data_ip[32];
    snprintf(data_ip, sizeof(data_ip), "%d.%d.%d.%d", h1, h2, h3, h4);
    int data_port = (p1 * 256) + p2;
    return connect_to_server(data_ip, data_port);
}

int main() {
    char buf[BUFFER_SIZE];
    int ctrl_sock = connect_to_server("lebavui.io.vn", 21);
    read_response(ctrl_sock, buf);

    send_cmd(ctrl_sock, "USER user_20235382\r\n");
    read_response(ctrl_sock, buf);
    send_cmd(ctrl_sock, "PASS 538205\r\n");
    read_response(ctrl_sock, buf);

    send_cmd(ctrl_sock, "PASV\r\n");
    read_response(ctrl_sock, buf);
    int data_sock_list = open_data_connection(buf);

    send_cmd(ctrl_sock, "LIST\r\n");
    read_response(ctrl_sock, buf);

    char list_output[8192] = {0};
    int total_list_bytes = 0;
    int bytes;
    while ((bytes = recv(data_sock_list, buf, sizeof(buf) - 1, 0)) > 0) {
        buf[bytes] = '\0';
        strcat(list_output, buf);
    }
    close(data_sock_list);
    read_response(ctrl_sock, buf);
```

```c
        char question_file[256] = {0};
        char answer_file[256] = {0};
        char *q_ptr = strstr(list_output, "question_");
        if (q_ptr) {
            sscanf(q_ptr, "%s", question_file);
            sprintf(answer_file, "answer_%s", question_file + 9);
            printf("\n[file founded] %s -> create answer file %s\n\n",
question_file, answer_file);
        } else {
            printf("\nError: Question file not found.\n");
            close(ctrl_sock);
            return 1;
        }

        send_cmd(ctrl_sock, "PASV\r\n");
        read_response(ctrl_sock, buf);
        int data_sock_retr = open_data_connection(buf);

        char retr_cmd[512];
        snprintf(retr_cmd, sizeof(retr_cmd), "RETR %s\r\n", question_file);
        send_cmd(ctrl_sock, retr_cmd);

        char file_content[1024] = {0};
        int total_content_bytes = 0;
        while ((bytes = recv(data_sock_retr, file_content + total_content_bytes,
sizeof(file_content) - total_content_bytes - 1, 0)) > 0) {
            total_content_bytes += bytes;
        }
        close(data_sock_retr);
        read_response(ctrl_sock, buf);
```

```
        while (total_content_bytes > 0 && (file_content[total_content_bytes-1]
== '\n' || file_content[total_content_bytes-1] == '\r')) {
            file_content[total_content_bytes-1] = '\0';
            total_content_bytes--;
        }

        char reversed_content[1024] = {0};
        for (int i = 0; i < total_content_bytes; i++) {
            reversed_content[i] = file_content[total_content_bytes - 1 - i];
        }
        printf("\nQuestion: %s\n", file_content);
        printf("Answer: %s\n\n", reversed_content);

        send_cmd(ctrl_sock, "PASV\r\n");
        read_response(ctrl_sock, buf);
        int data_sock_stor = open_data_connection(buf);

        char stor_cmd[512];
        snprintf(stor_cmd, sizeof(stor_cmd), "STOR %s\r\n", answer_file);
        send_cmd(ctrl_sock, stor_cmd);

        send(data_sock_stor, reversed_content, total_content_bytes, 0);
        close(data_sock_stor);

        read_response(ctrl_sock, buf);

        send_cmd(ctrl_sock, "QUIT\r\n");
        read_response(ctrl_sock, buf);

        close(ctrl_sock);
        return 0;
    }
```